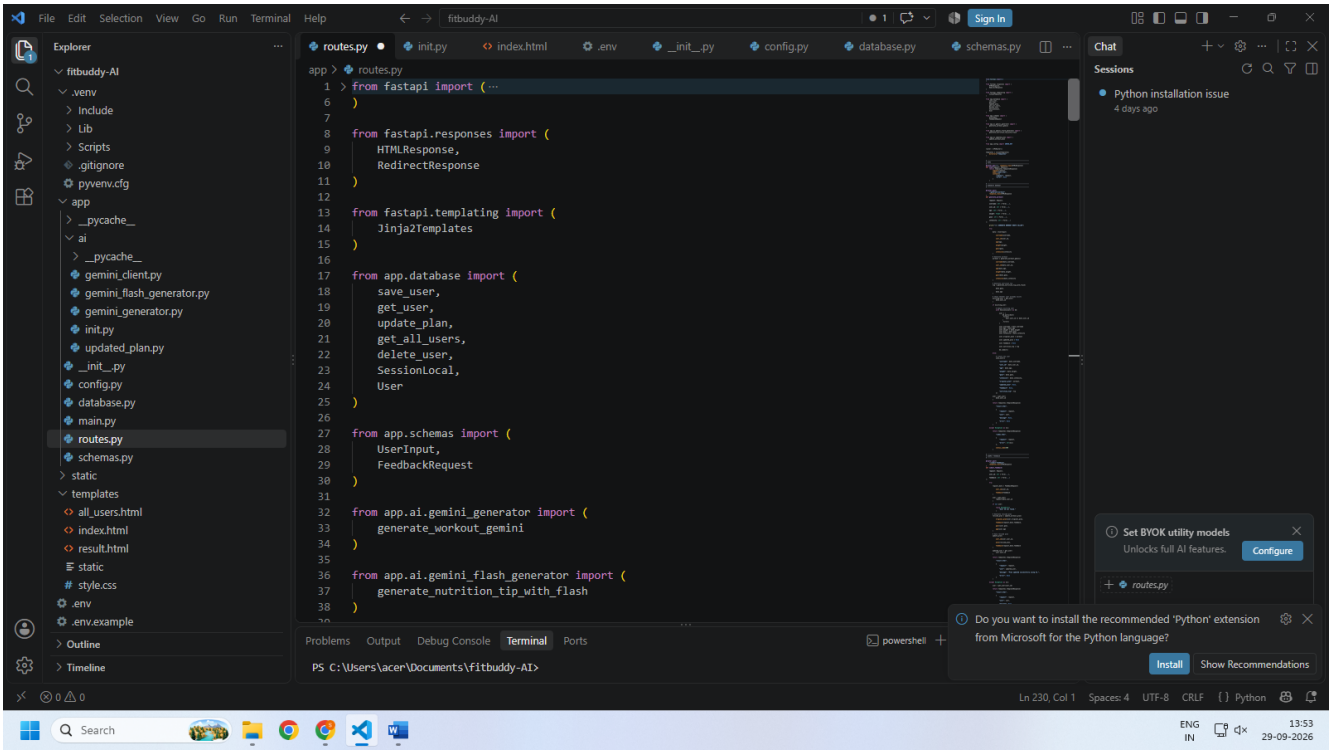
FitBuddy-AI Fitness Plan Generator using Gemini Models

1. Introduction

The development phase converts the design into an executable FitBuddy application. The implementation combines FastAPI routing, HTML templates, SQLAlchemy database operations, Gemini AI generation, form handling, feedback processing, and administrative features.

2. FastAPI Backend Development

FastAPI is used as the backend framework. The router contains the main application endpoints. The home endpoint returns index.html. The generate-workout endpoint receives the form fields and creates a UserInput object. The submit-feedback endpoint receives user_id and feedback and uses the stored user information to generate a revised plan.



The view-all-users endpoint displays stored users when a valid admin key is supplied. The delete-user endpoint removes a selected user after administrator validation. The health endpoint provides a lightweight service-status response.

The application is executed using Uvicorn. Reload mode is useful during development because changes in the source files can trigger a server reload.

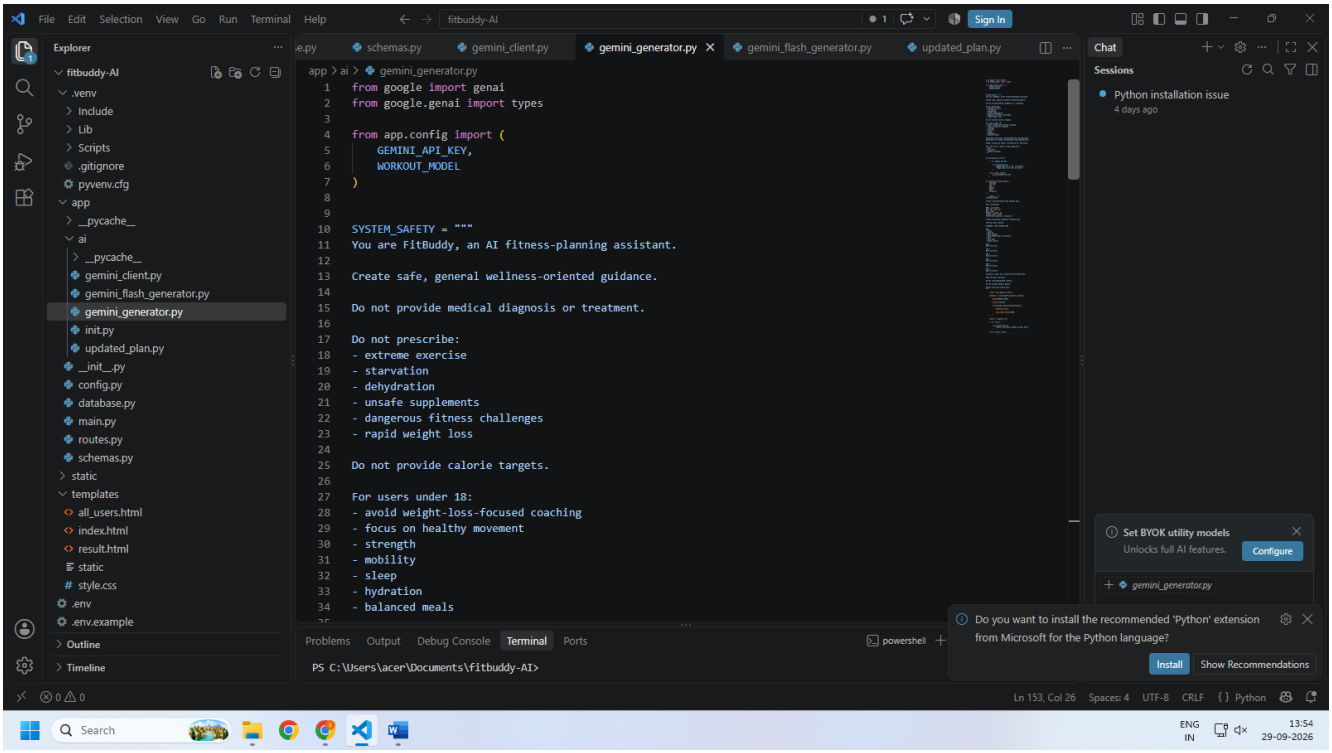
3. Form Processing and Validation

The project uses FastAPI Form parameters for HTML form submissions. The fields username, user_id, age, weight, goal, and intensity are collected and passed into the UserInput schema. Feedback is passed through FeedbackRequest.

The use of schemas provides a structured representation of user data before the AI and database functions process it. The project also requires python-multipart because FastAPI needs multipart/form parsing support for form-based requests.

4. Gemini AI Integration

The workout generation module receives the user's basic fitness information and sends the relevant content to Gemini through the Google GenAI interface. The purpose is to generate a personalized workout response rather than returning a fixed hard-coded plan.

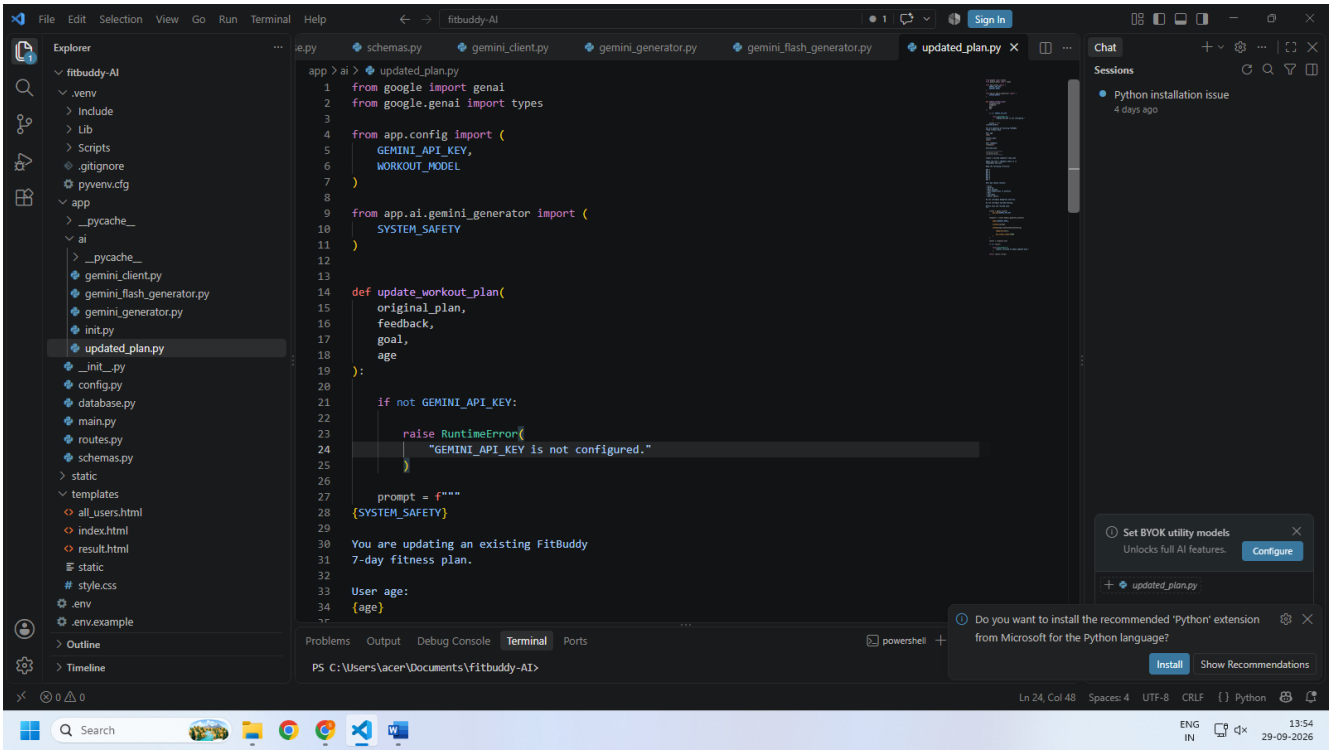


A separate Gemini-based nutrition function generates a nutrition tip based on the user's goal and age. Separating these functions makes the application easier to maintain and allows different generation logic to be changed independently.

5. Feedback-Based Plan Update

Feedback is one of the main interactive features of FitBuddy. When a user submits feedback, the system first searches for the user by user ID. If the user does not exist, the application returns an error. If the user exists, the original plan, feedback, goal, and age are sent to update_workout_plan.

The revised plan is stored as updated_plan and the feedback is stored in the user record. The updated record is then retrieved and displayed through result.html. This provides a clear before-and-after workflow for demonstration.



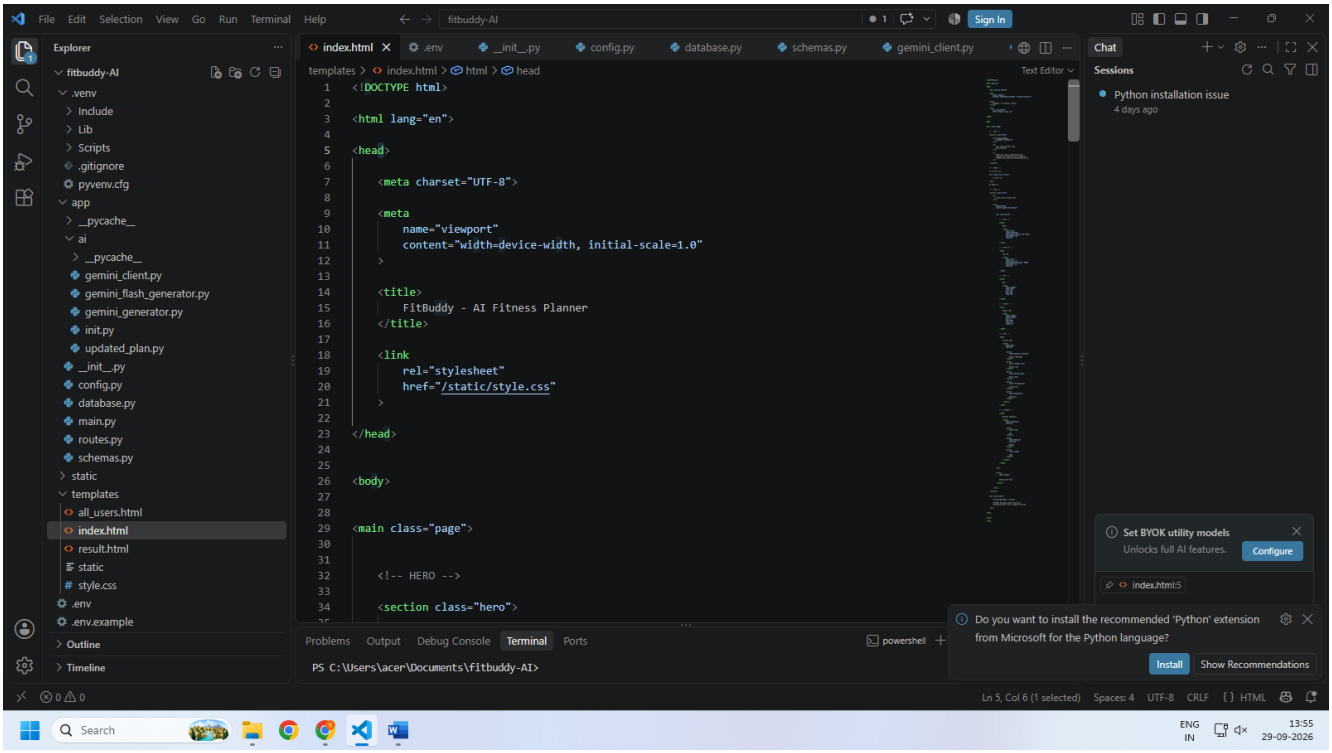
6. Database Development

SQLite is used for local storage and SQLAlchemy is used to interact with the database. The User model represents the stored user information. Functions such as save_user, get_user, update_plan, get_all_users, and delete_user provide reusable database operations.

The existing-user logic prevents unnecessary duplicate records when the same user ID is submitted again. Instead, the existing record is updated with the latest information and generated plan.

7. Frontend and Templates

Jinja2Templates is used to render the HTML files. index.html is the main input page, result.html is the output and feedback page, and all_users.html supports administrator viewing.



The templates are connected to the FastAPI routes through TemplateResponse. A static directory is also configured so that CSS and other static resources can be served separately from the template files.

8. Conclusion

The development phase integrates all planned components into a working prototype. The completed implementation demonstrates AI generation, database persistence, web routing, feedback-based updating, and administration. The result is a full workflow from user input to AI-generated output and revised output.

Phase Deliverables / Evidence

Item	Purpose	Status / Evidence
Backend	Implements FastAPI routes	Implemented
AI integration	Connects Gemini modules	Implemented
Database/UI	Stores data and renders pages	Implemented